

Chapter 3: Building a Simple Fully Connected Neural Network

Objective: To implement a simple fully connected neural network (FCNN) using TensorFlow and Keras for classifying images in the CIFAR-10 dataset. This session will guide you in defining the architecture, setting up training, and running the model.

1. Define a Simple Neural Network Architecture

Tasks:

1. Use **Keras Sequential API** to define the structure of your fully connected neural network.
2. Your model should include:
 - **Input Layer:** This layer will accept flattened images (32x32x3 reshaped to a vector of 3072).
 - **Hidden Layers:** Add 1-2 dense (fully connected) layers with ReLU activation.
 - **Output Layer:** Add a dense layer with 10 neurons (for 10 classes) and softmax activation.
3. Ensure you include `Flatten()` as the first layer to convert 3D image arrays into 1D vectors.

```
In [2]: import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense
from sklearn.model_selection import train_test_split

# 1. Load the CIFAR-10 dataset
# Use the dataset prepared in Chapter 2 (train, validation, and test splits)
(train_images_full, train_labels_full), (test_images,
                                         test_labels) = tf.keras.datasets.cifar10.load_data()

# Normalize pixel values to be between 0 and 1
train_images_full = train_images_full / 255.0
test_images = test_images / 255.0

# Perform train and validation split (e.g., 80% train, 20% validation)
train_images, val_images, train_labels, val_labels = train_test_split(
    train_images_full, train_labels_full, test_size=0.2, random_state=42
)

# 2. Define the model architecture
model = Sequential([
    # First Layer: Flatten, reshapes the image arrays into a 1D vector (size 32*32*3)
    Flatten(input_shape=(32, 32, 3)),

    # Hidden Layers: Add Dense Layers to Learn features, using ReLU activation f
```

```

Dense(128, activation='relu'),
Dense(64, activation='relu'),

# Output Layer: CIFAR-10 has 10 classes, so we use 10 neurons with softmax a
Dense(10, activation='softmax')
])

# Print the model architecture summary
model.summary()

```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
flatten_1 (Flatten)	(None, 3072)	0
dense_3 (Dense)	(None, 128)	393,344
dense_4 (Dense)	(None, 64)	8,256
dense_5 (Dense)	(None, 10)	650

Total params: 402,250 (1.53 MB)

Trainable params: 402,250 (1.53 MB)

Non-trainable params: 0 (0.00 B)

Questions to Explore:

- Why does the input layer require a `Flatten()` operation for image data?
- What does the softmax activation function do in the output layer, and why is it important for classification?

Answer:

1. **Purpose of `Flatten()`** : Dense layers only accept 1D arrays. `Flatten()` reshapes the 3D image (32x32x3) into a 1D vector (size 3072) so it can be fed into the network.
2. **Role of Softmax**: It converts raw model outputs into a probability distribution (values between 0 and 1 that sum up to 1). This clearly shows the model's predicted probability for each of the 10 classes.

2. Compile the Model

Tasks:

1. Compile your model using the following:
 - **Loss Function**: Use `sparse_categorical_crossentropy` for multi-class classification.
 - **Optimizer**: Use `Adam` or experiment with `SGD`.

- **Metrics:** Include `accuracy` as a metric to evaluate performance during training.
2. What does compiling a model mean in TensorFlow? Write a short explanation in your notes.

```
In [3]: # 3. Compile the model
# Your code here
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

Answer:

2. **Compiling a Model:** It means configuring the learning process before training starts. You set up the **Optimizer** (how weights update), **Loss Function** (what the model tries to minimize), and **Metrics** (how to monitor performance like accuracy).

3. Train the Model

Tasks:

1. Use the `model.fit()` function to train the FCNN on the training dataset.
 - Pass the training images and labels, and specify the batch size and number of epochs.
 - Use your validation set to monitor performance during training.
2. Plot the training and validation accuracy over epochs. What do you observe? Is the model overfitting or underfitting?

```
In [8]: import matplotlib.pyplot as plt
# 4. Train the model
history = model.fit(
    train_images,
    train_labels,
    epochs=20,
    batch_size=64,
    validation_split=0.2 # Using automatically split validation set
)

# 5. Plot training and validation accuracy
# extract 'accuracy' and 'val_accuracy' from the history dictionary
plt.figure(figsize=(8, 6))
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

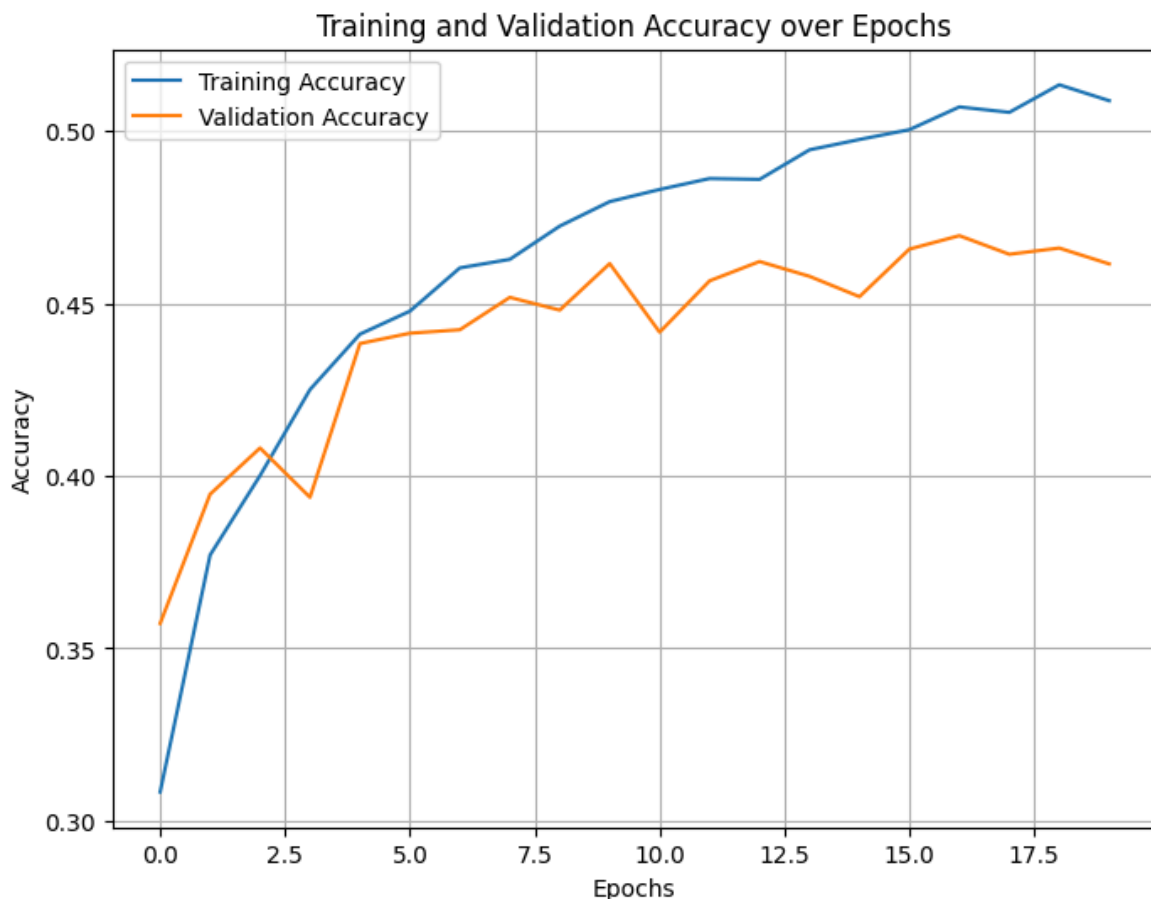
# Adding labels, title, and legend to the plot
plt.title('Training and Validation Accuracy over Epochs')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
```

```
plt.grid(True)  
plt.show()
```

```

Epoch 1/20
625/625 ————— 5s 5ms/step - accuracy: 0.2568 - loss: 2.0338 - val_
accuracy: 0.3572 - val_loss: 1.8054
Epoch 2/20
625/625 ————— 2s 3ms/step - accuracy: 0.3658 - loss: 1.7591 - val_
accuracy: 0.3947 - val_loss: 1.6975
Epoch 3/20
625/625 ————— 2s 3ms/step - accuracy: 0.3992 - loss: 1.6643 - val_
accuracy: 0.4081 - val_loss: 1.6643
Epoch 4/20
625/625 ————— 2s 3ms/step - accuracy: 0.4241 - loss: 1.6099 - val_
accuracy: 0.3938 - val_loss: 1.6956
Epoch 5/20
625/625 ————— 3s 4ms/step - accuracy: 0.4397 - loss: 1.5785 - val_
accuracy: 0.4384 - val_loss: 1.5734
Epoch 6/20
625/625 ————— 4s 3ms/step - accuracy: 0.4448 - loss: 1.5426 - val_
accuracy: 0.4414 - val_loss: 1.5802
Epoch 7/20
625/625 ————— 2s 3ms/step - accuracy: 0.4585 - loss: 1.5156 - val_
accuracy: 0.4424 - val_loss: 1.5669
Epoch 8/20
625/625 ————— 2s 4ms/step - accuracy: 0.4650 - loss: 1.4928 - val_
accuracy: 0.4518 - val_loss: 1.5615
Epoch 9/20
625/625 ————— 2s 4ms/step - accuracy: 0.4757 - loss: 1.4839 - val_
accuracy: 0.4481 - val_loss: 1.5596
Epoch 10/20
625/625 ————— 3s 4ms/step - accuracy: 0.4829 - loss: 1.4574 - val_
accuracy: 0.4616 - val_loss: 1.5115
Epoch 11/20
625/625 ————— 2s 3ms/step - accuracy: 0.4906 - loss: 1.4379 - val_
accuracy: 0.4417 - val_loss: 1.5957
Epoch 12/20
625/625 ————— 2s 4ms/step - accuracy: 0.4868 - loss: 1.4305 - val_
accuracy: 0.4566 - val_loss: 1.5337
Epoch 13/20
625/625 ————— 2s 3ms/step - accuracy: 0.4870 - loss: 1.4260 - val_
accuracy: 0.4622 - val_loss: 1.5087
Epoch 14/20
625/625 ————— 2s 3ms/step - accuracy: 0.4941 - loss: 1.4194 - val_
accuracy: 0.4579 - val_loss: 1.5250
Epoch 15/20
625/625 ————— 3s 4ms/step - accuracy: 0.4979 - loss: 1.4038 - val_
accuracy: 0.4520 - val_loss: 1.5723
Epoch 16/20
625/625 ————— 2s 3ms/step - accuracy: 0.5007 - loss: 1.3964 - val_
accuracy: 0.4658 - val_loss: 1.5121
Epoch 17/20
625/625 ————— 2s 3ms/step - accuracy: 0.5129 - loss: 1.3804 - val_
accuracy: 0.4697 - val_loss: 1.5077
Epoch 18/20
625/625 ————— 2s 3ms/step - accuracy: 0.5071 - loss: 1.3814 - val_
accuracy: 0.4643 - val_loss: 1.5332
Epoch 19/20
625/625 ————— 2s 3ms/step - accuracy: 0.5173 - loss: 1.3586 - val_
accuracy: 0.4661 - val_loss: 1.5213
Epoch 20/20
625/625 ————— 2s 3ms/step - accuracy: 0.5074 - loss: 1.3729 - val_
accuracy: 0.4615 - val_loss: 1.5186

```

**Answer:**

1. **Observation:** Training accuracy keeps increasing, but validation accuracy quickly stops improving and hits a plateau.
2. **Overfitting:** This gap shows the model is overfitting. It memorized the training data's noise but fails to generalize well to unseen validation data.

4. Evaluate the Model

Tasks:

1. Evaluate your trained model on the test set using `model.evaluate()`.
2. Report the test accuracy. How does it compare to the training and validation accuracy?
3. Identify possible reasons if the test accuracy is significantly lower than the training accuracy. What might be done to improve it?

```
In [9]: # 6. Evaluate the model on the test set
# model.evaluate() calculates the loss and metrics we defined earlier (accuracy)
# on a given dataset (in this case, our test_images and test_labels).
test_loss, test_accuracy = model.evaluate(test_images, test_labels, verbose=2)

# Print the evaluation results in a readable format
```

```
print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")
```

313/313 - 2s - 6ms/step - accuracy: 0.4736 - loss: 1.5021

Test Loss: 1.5021

Test Accuracy: 47.36%

Answer:

1. **Comparison:** Test accuracy is similar to the validation accuracy but significantly lower than the final training accuracy.
2. **Reasons:** The model is overfitting. Also, a Fully Connected Network (FCNN) loses 2D spatial context of images because of the flattening step.
 - **Improvements:** Switch to a **Convolutional Neural Network (CNN)**. Use regularization (like Dropout) or Data Augmentation.

Reflection Questions:

1. How did you decide on the number of layers and neurons in your FCNN?
2. What observations did you make from the training and validation accuracy plots?
3. How does the test accuracy compare to the training and validation accuracy? What might explain these differences?
4. What changes would you consider making to improve the performance of your FCNN?

Answer:

1. **Architecture:** Hidden layers needed enough neurons (e.g., 128, 64) to learn features. The output layer must have 10 neurons to match CIFAR-10's 10 classes.
2. **Plots:** The growing gap between the training and validation lines clearly shows the model stops generalizing and starts memorizing early on.
3. **Comparison:** Test accuracy is much lower than training accuracy. This disparity is mainly due to **overfitting**.
4. **Improvements:** FCNNs destroy image structures by flattening them. The best fix is to use a **CNN (Convolutional Neural Network)**, and add techniques like **Dropout** or **Data Augmentation**.